

Лабораторна робота №4

Проект “impulse”

1. Тема і мета роботи:

Тема: рефакторинг коду та проведення code review за індустріальними стандартами

Мета роботи: Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests.

Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

2. Результати Code Review одnogрупника:

	Рядок коду	Проблема	Категорія	Рекомендація
1	7 (// Метод 1: Розрахунок Індексу Маса Тіла (ІМТ))	Зайві коментарі є надлишковими. Код має бути самодокументованим.	Bad Comments	Видалити всі пояснювальні коментарі.
2	25 (gender.ToLower())	Метод gender.ToLower() викликається кілька разів.	Duplicated Code	Викликати його один раз на початку методу і зберегти у змінну (наприклад, var normalizedGender = gender.ToLower();).
3	27 (88.36 + (13.4...))	Коефіцієнти формули прописані прямо в логіці.	Magic Numbers	Винести їх у private const double константи класу.

4	29 ("female", "жінка")	Перевірка за допомогою текстових рядків може призвести до помилок.	Magic Strings	Замінити на enum Gender { Male, Female }
5	50 (/ 30.0) * 0.5)	Значення використані у формулі без пояснення	Magic Numbers	Винести в константи зі зрозумілими назвами.

3. Посилання на PR з коментарями Code Review (URL).

<https://github.com/yelyzavetaholovko-ui/Impulse/pull/2>

4. Результати рефакторингу власного коду:

1. Операція 1: Заміна магічних рядків на перерахування (Replace Magic Strings with Enum)

БУЛО: `double met = activityType.ToLower() switch { "біг" => 8.0, "плавання" => 7.0, "силове" => 5.0, _ => 3.0 };`

СТАЛО: `double met = activity switch { ActivityType.Running => RUNNING_MET, ActivityType.Swimming => SWIMMING_MET, ActivityType.Strength => STRENGTH_MET, _ => DEFAULT_MET };`

ЧОМУ: Використання звичайних рядків (string) для типу активності замість жорсткої типізації є небезпечним, оскільки легко допустити друкарську помилку. Заміна на enum ActivityType робить код безпечнішим і перевіряється на етапі компіляції.

2. Операція 2: Винесення магічних чисел у константи (Replace Magic Number with Symbolic Constant)

БУЛО: `int maxHeartRate = 220 - age;`

СТАЛО: `private const int BASE_MAX_HEART_RATE = 220; int maxHeartRate = BASE_MAX_HEART_RATE - age;`

ЧОМУ: Число 220 використовувалося прямо в коді як незмінна база для розрахунку пульсу без пояснення контексту. Винесення його у константу BASE_MAX_HEART_RATE робить формулу зрозумілою та полегшує подальшу підтримку коду.

3. Операція 3: Видалення мертвого коду (Remove Dead Code)

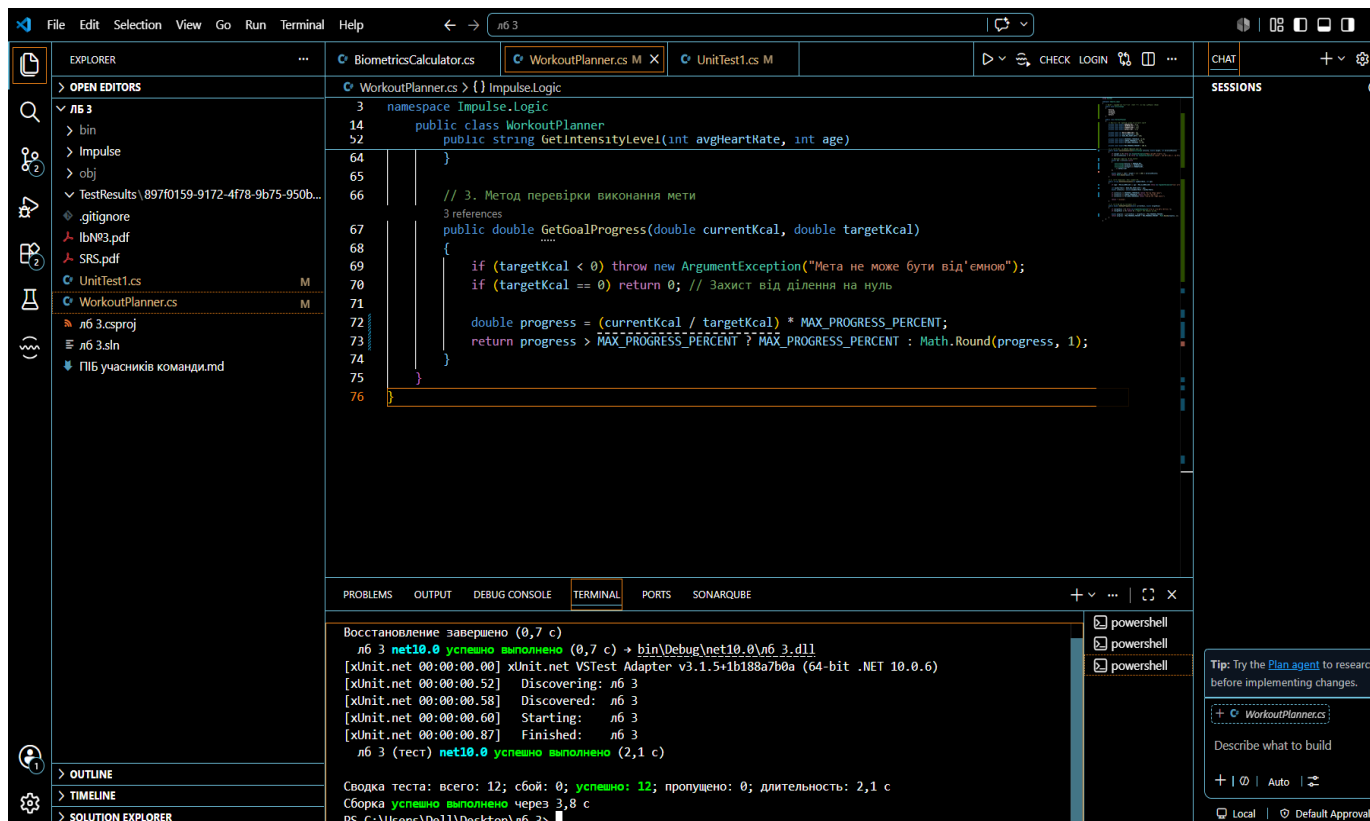
БУЛО: class Program { static void Main(string[] args) { } }

СТАЛО: Порожній клас Program та метод Main повністю видалено з файлу WorkoutPlanner.cs.

ЧОМУ: Наявність порожнього класу та методу, які ніколи не виконуються, є "мертвим кодом", що засмічує проєкт. Проблема вирішено архітектурно (змінюю тип проєкту на бібліотеку класів у файлі .csproj), після чого непотрібний код було безпечно видалено.

5.Звіт регресійного тестування: скріншот результату pytest / JUnit / Jest усі тести green після рефакторингу.

Звіт регресивного тестування



The screenshot displays the Visual Studio IDE with the following components:

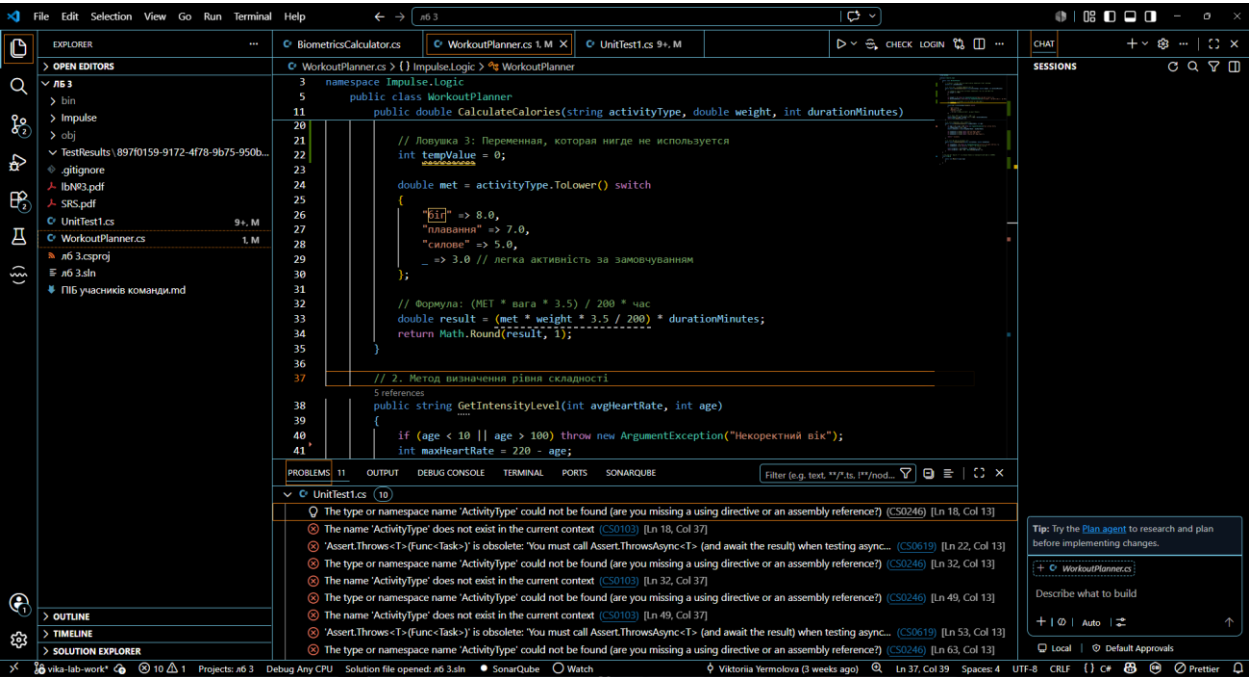
- EXPLORER:** Shows the project structure with folders like 'bin', 'Impulse', and 'obj'. The file 'WorkoutPlanner.cs' is selected.
- WORKOUTPLANNER.CS:** The main code file is open, showing the 'Impulse.Logic' namespace and the 'WorkoutPlanner' class. The code includes methods for calculating goal progress and intensity levels.
- TERMINAL:** Displays the output of a test run using xUnit.net. The output shows that all tests passed successfully.

```
Восстановление завершено (0,7 с)
лб 3 net10.0 успешно выполнено (0,7 с) → bin\Debug\net10.0\лб 3.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.5+1b188a7b0a (64-bit .NET 10.0.6)
[xUnit.net 00:00:00.52] Discovering: лб 3
[xUnit.net 00:00:00.58] Discovered: лб 3
[xUnit.net 00:00:00.60] Starting: лб 3
[xUnit.net 00:00:00.87] Finished: лб 3
лб 3 (тест) net10.0 успешно выполнено (2,1 с)

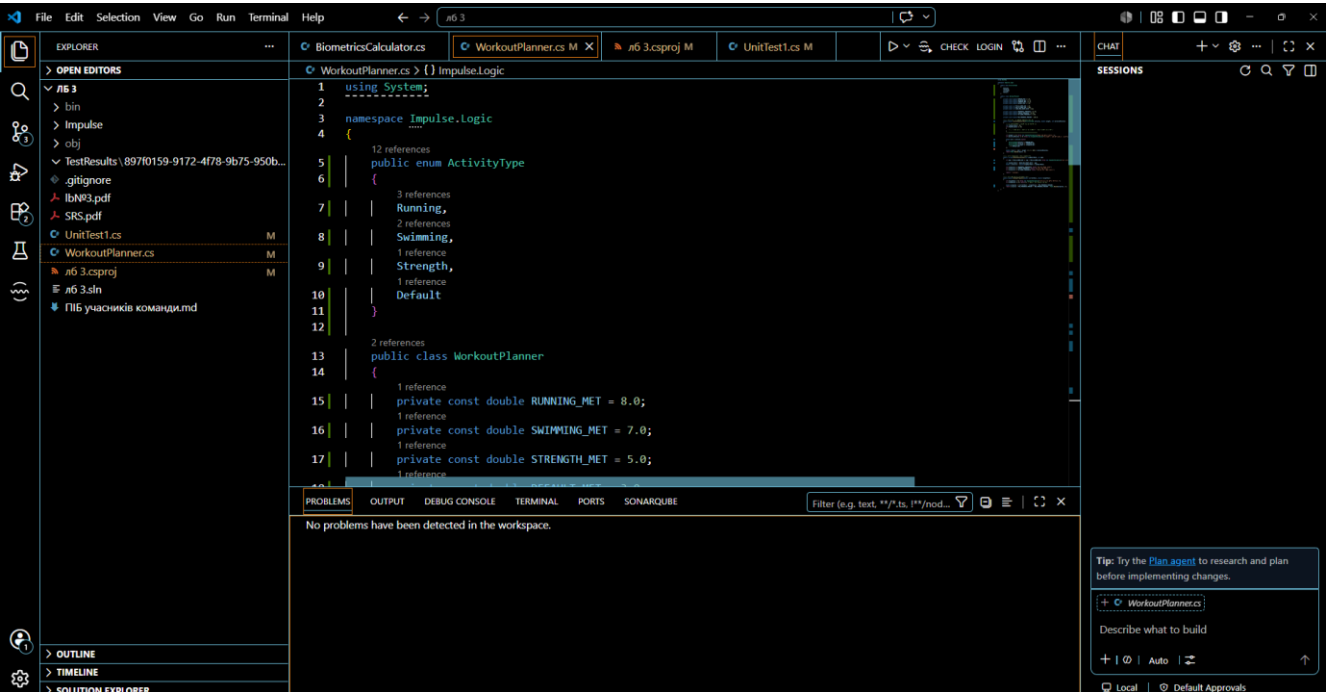
Сводка теста: всего: 12; сбой: 0; успешно: 12; пропущено: 0; длительность: 2,1 с
Сборка успешно выполнено через 3,8 с
PS C:\Users\ DELL\Desktop\лб 3>
```

6. Порівняння метрик «ДО / ПІСЛЯ»: кількість SonarLint issues, Ruff/ESLint warnings, цикломатична складність.

До:



Після:



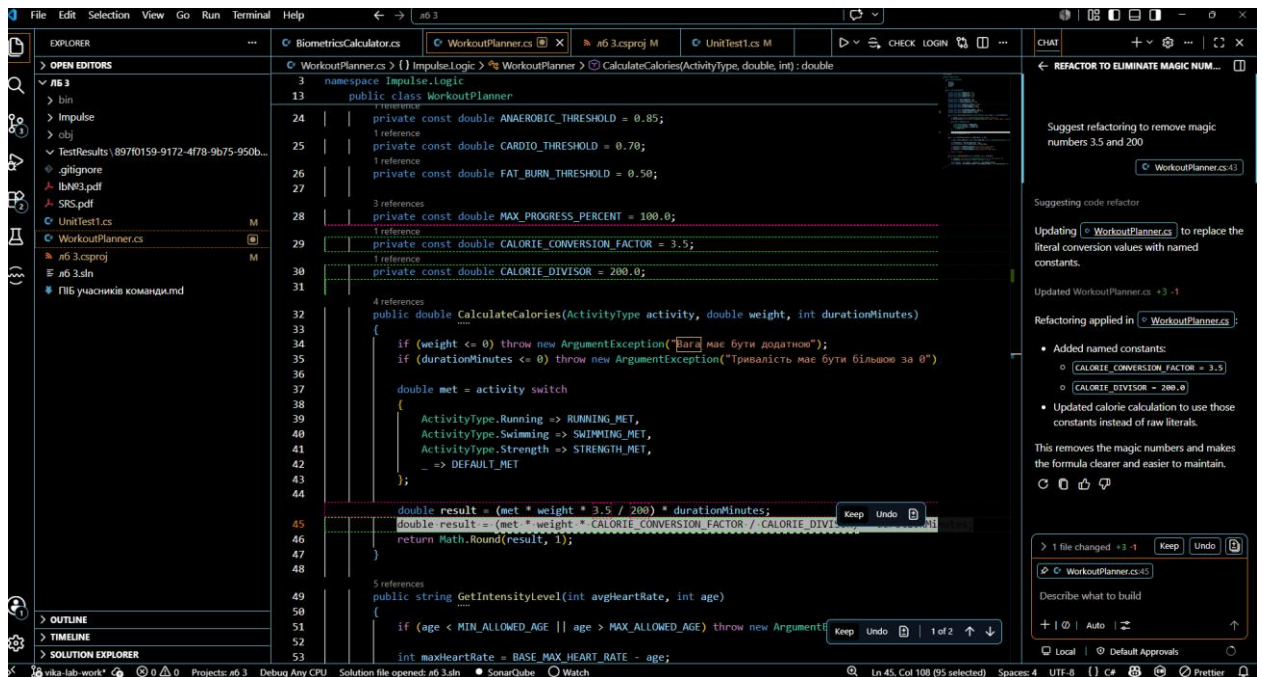
Цикломатична складність:

ДО рефакторингу: 16 (загальна).

ПІСЛЯ рефакторингу: 16 (загальна).

Обґрунтування: Показник цикломатичної складності не змінився, оскільки проведені операції рефакторингу (Replace Magic Strings with Enum, Replace Magic Number with Symbolic Constant, Remove Dead Code) стосувалися створення безпечних перерахунків, винесення констант та видалення класу, який не використовувався. Ці дії оптимізували структуру даних, але не змінили кількість логічних розгалужень (операторів if та switch) у методах.

***Бонусне завдання:** використати GitHub Copilot Chat для отримання пояснення підозрілого фрагмента коду (Explain this code) або пропозиції рефакторингу (Suggest refactoring). Результат включити у звіт.



систенту (AI Chat) було надіслано запит на рефакторинг фрагмента коду з формулою розрахунку спалених калорій, де залишалися "магічні числа" (3.5 та 200). ШІ розпізнав архітектурну проблему (Magic Numbers) та успішно запропонував винести ці числові значення у константи з описовими іменами (CALORIE_CONVERSION_FACTOR та CALORIE_DIVISOR) для покращення читабельності та підтримки коду. Результат виконання запиту та згенероване ШІ пояснення наведено на скріншоті.

7. Підсумкова рефлексія:

Найціннішим досвідом під час виконання цієї лабораторної роботи стало усвідомлення того, наскільки критично важливим є системний підхід до рефакторингу. Спільна перевірка коду (Code Review) з напарницею дозволила виявити архітектурні недоліки, такі як «магічні числа» та надмірна складність методів, які не завжди фіксуються стандартними лінтерами.

Працюючи з інструментами штучного інтелекту (GitHub Copilot Chat), я навчилася не лише швидше знаходити шляхи оптимізації коду, але й критично оцінювати пропоновані ШІ рішення. Цей досвід довів, що своєчасний рефакторинг та використання сучасних практик (використання `enum`, констант, модульних тестів) є не просто «прикрашанням» коду, а фундаментальною основою для його надійності, читабельності та легкості майбутнього обслуговування. Робота з Git-репозиторіями та Pull Requests остаточно закріпила розуміння того, як організована розробка виглядає в реальних командних проектах.